Thinking Beyond Unit Testing

This article is intended for those who want to verify and improve code quality, and to delve into unit testing.

Unit testing is a highly effective verification and validation technique in software engineering. You can use unit testing to improve code quality. In this article, we discuss unit testing using the UnitTest++ tool. We explore how to decipher code coverage using lcov, and then move on to valgrind to check for memory leaks.

Prerequisites

You need to install *UnitTest++*, *lcov* and *valgrind*. The compilation and installation process mainly needs GCC, g++ and Perl on your system. I have successfully installed these tools under Fedora 10 and RHEL5. In Ubuntu 9.10, I had to manually install g++. If you get into dependency trouble during installation, I would recommend that you use the package management software of your distribution to install these packages and their dependencies. The commands for those would be: *yum install <packagename>* (Redhat/Fedora); *apt-get install <packagename>* (Debian/Ubuntu); *zypper install <packagename>* (OpenSuSE); and *urpmi <packagename>* (Mandriva).

Let's assume that you are going to install the tools in your *$HOME/tools* directory, and that your source and test code is in the *$HOME/src* and *$HOME/test* directories, respectively. If this is not the case, use the paths that are specific to your system, while setting up the environment variables below.

> **Note:** Your login account should have *sudo* privileges to execute *make install*, as in the command snippets below. Alternately, you can run the commands as the *root*.

1. Export the paths to your folders as environment variables:

```
bash> export TOOLSROOT=$HOME/tools
bash> export SRCROOT=$HOME/src
bash> export TESTROOT=$HOME/test
```